

1 Java Syntax

- **Statically-typed** = **declare variables**
 - ↳ `int a = 5;`
 - ↳ `int[] b = new int[10];`
 - ↳ `car Toyota = new Car;`
 - (Declaration) (Initialization)

2 Types

- **Primitive** = bytes stored at location of variable in memory (boolean, int, char, double) → aka float/double
- **Reference** = memory address stored at location & points to object (Strings, Arrays, LinkedLists, Dogs)
- **BoRE**: "copies all bits"

```

int x = 5; xE
int y = x; yE

Cat c = new Cat();
cat e = c;

int[] arr = new int[] {1, 2, 3}
arr L [12]
    
```

3 class structure

```

public class CS61BStudent {
    public int idNumber;
    public int grade;
    public static String instructor = "Justin";
    public CS61BStudent(int id) { ← constructor
        this.idNumber = id;
        this.grade = 100;
    }
    public boolean watchLecture() { ← instance method called on instance
        ...
    }
    public static String getInstructors() { ← keyword
        ...
    }
}
    
```

class (static) var.

return type: void = nothing

parameters

method body

method name

public static void main(String[] args)

4 scope

- ↳ **private** = variable/method accessible to that class only
- ↳ **public** = accessible by ALL other classes

5 Conditional

- ↳ if, else if, else
- ↳ && (and), || (or), ! (not), ==
- ↳ true, false

identity equals() for equality & reference types

6 While loops

```

int i = 0;
while (i < 10) {
    system.out.println(i);
    i++;
}
    
```

Initialization: while (termination) { body increment

7 for loops

```

for (int i = 0; i < 10; i++) {
    system.out.println(i);
}
    
```

initialize terminate increment

8 foreach loop

```

for (String s : arr) {
    system.out.println(s);
}
    
```

Lists & Sets work must implement Iterable

9 Sets

- `Set<Integer> set = new HashSet<>()`
- `Set.add(value)` & `Set.remove(value)`
- no duplicate items
- foreach loop ok!

treeSet = sorted

10 Maps (dictionaries)

```

Map<String, String> map = new HashMap<>();
map.put(key, value);
map.get(key) → value;
map.containsKey(key) → bool;
for (String key : map.keySet()) ...
    
```

treeMap w/ sorted keys

foreach OK! w/ keySet

4 Static & Instance

- **Static** - belong to whole class
 - ↳ every CS61BStudent shares same prof
- **Instance** - belong to individual instance
 - ↳ own ID number; change one doesn't change another

5 This vs. Static

- **this**: non-static method only called using instance of that object; access to it using this.
- **Static method**: don't require instance of that object to be called; can't rely on access
 - ↳ can't run instance var. raw; must be referenced
- **Static variables**: shared by all instances; instance w/o own copy, but can access
- **Instance method**: can run static or instance

6 Arrays (fixed)

- fixed length & same type of value
- `int[] zeroedArray = new int[5]` → [0, 0, 0, 0, 0]
- `int[] array = {4, 1, 10}` → [4, 1, 10]
- declare, assign, instantiate
- `arr[index] → retrieve element @ index`
- `arr[index] = value → update elem. @ index to value`
- `arr.length → # of elem.`
- `Arrays.toString(arr) → print`
- can use for or for each

index out of bounds Err.

retrieve direct

7 Array Lists (lists, resizable)

- implements List, extend Collections
- `List<Obj> = new ArrayList<Obj>()` (i.e. <String>)
- `List<String> list = new ArrayList<>()`
- `list.add(value)`
- `list.get(index)`
- `list.set(index, value)`
- `list.remove(index)`
- `list.size()`
- `list.isEmpty()`
- `list.equals(object) ← same elem/ order`
- `list.contains(object) ← bool`
- `list.set(index, value) ← update`

8 Strings

```

String s = "hello";
s += " world";
s.length() → 12
s.substring(1, 5) → "ello"
s.charAt(2) → 'l'
    
```

for (int i = 0; i < s.length(); i++) { char letter = s.charAt(i); }

iterate over an index & use charAt

9 Exceptions

- implicit: stop & crash
- throw & instantiate
 - ↳ `public void add(T x) { if (x == null) { throw new IllegalArgumentException("Bad!"); } }`

10 Junit

- `assertThat(list.isEmpty(), isTrue())`
- `assertThat(list.contains(0), isExactly(1))`
- `isEqualTo(-)`

4 Linked Lists

- recursive data structure
- first/item: element stored
- rest/next: point to next node

IntList

```

SLL that holds int
IntList x L [4] → [1] → [8] → [5]
x.next.next.item = 8
x.next.next.next = new Node(5)
x.next.next = null
    
```

Node

```

Node
item
next
    
```

one direction

5 Doubly Linked List (DLL)

Node

```

Node
item
next
prev
    
```

Traverse in both directions

6 Abstraction

- sentinel to easily add & delete nodes
 - ↳ prev = last node
 - ↳ next = first node

for each okay for Deque

Deque: DLL + sentinel

```

Deque
addLast()
getFirst()
...
    
```

arrayDeque

- ↳ nextFirst & nextLast
- ↳ resize by copying new array over

7 Nested classes

- classes w/in class
- help construct larger class (i.e. Node for DLL)
- public = anyone can access
- private = only you can access w/in class
- static = only uses variables w/in its (nested) class, method access class variables & methods
- instance = " " & instance var/methods

8 Inheritance

- Subclass: inherits parent's non-priv functions/vars + own funcn/vars
- Superclass: parent, to be inherited
- classes → extend
 - Superclass → subclass, only 1
 - "is-a"
 - Choose to override methods
 - How
 - default parent's constructor or call `super()`
 - Specify if multiple: `super(x)`
- Interfaces → implement
 - List → ArrayList, multiple OK!
 - may default implement methods
 - must override all non-default methods
 - cannot be instantiated
 - WHAT
- Method overloading
 - multiple methods w/ same name, diff para.
 - pick at RT based on object type
- Method overriding
 - same method signature in both
 - diff s & p → pick more specific as ST
 - "overriding" method from ST
 - ST doesn't have function → CE

9 Approach

- iteratively: while (...)
- traversing → recursively: if & else
- destructively: change in non-des. create new

10 Inheritance Cont.

- Compile time: read & don't run
- Run time: code execute
- Static Type - declaration
- Dynamic Type - instantiation (new)

Dog d = new Corgi();

Static Type CT

Dynamic Type RT

can hold anything that is-a Dog

11 Casting

- changes static type of variable temporarily
- Can cast up/down, not sideways
- CE: static type is not casted type
- RE: dynamic type follows "is-a" relationship w/ casted type
 - ↳ current DT should be subclass/same type as class we trying to cast to

12 DMS

- CT: static type of invoking instance
 - 1) check for valid variable assignments
 - 2) check for valid method calls
 - consider static type & ST superclass(es)
 - lock-in exact method signature, traversing parents
 - 3) Nothing → CE
- RT: dynamic type of invoking instance
 - 1) if locked-in method is static → run it
 - 2) Else, ensure casted objects can be assigned to their variables
 - 3) check for overridden methods
 - start w/ dynamic, if overridden method → run it
 - or traverse up

13 Variable Assignment

```

graph TD
    A["X = B Y; Is B in super-sub relationship w/ Y's ST?"] -->|Y| B["OK!"]
    A -->|N| C["Is B = A or subclass of A?"]
    C -->|Y| B
    C -->|N| D["Is DT of Y B or subclass of B?"]
    D -->|Y| B
    D -->|N| E["RE"]
    
```

casting?

CE

RE

14 Iterator

```

public interface Iterator<T> {
    boolean hasNext();
    T next();
}
    
```

boolean hasNext() → another elem to iterate over?

T next() → give next elem

noSuchElementException

Iterator = the thing that does the iterating, customize iteration

ie public Iterator<Dog> iterator()

Iterable

```

public interface Iterable<T> {
    Iterator<T> iterator();
}
    
```

Iterable = the thing that gets iterated over

- If class is Iterable → can use for-each (ie Lists)
- object that can produce iterator

for (String x : listOfStrings)

↳ for (Iterator<String> iter : listOfStrings.iterator(); iter.hasNext(); someObject x = iter.next();)

15 == vs. equals()

- primitive:
 - a L a == b
 - b L a.equals(b) not allowed
- reference:
 - a L Dog
 - b L Dog
 - c L Dog

a == b

a != c b/c diff instances

a.equals(b)

a.equals(c)

our def. of same

- 17) Comparable/Comparator**
- comparable
 - to compare objects/reference types
 - Implement Comparable<T>
 - o1.compareTo(o2)
 - Override int compareTo(T object)
 - ⊖ object less, ⊕ object equal, ⊕ object greater than specified/given object - this
 - comparators ← see saw
 - objects that let us compare 2 objects
 - Implement Comparator<T>
 - public int compare(T obj1, T obj2)
 - ⊖ obj1 less, ⊕ obj1 equal, ⊕ obj1 greater
 - ↳ can compare by diff things

- 18) Recursion**
- recursive call patterns
 - processing integers digit by digit: myFunc(n/10)
 - until an integer reaches 0: myFunc(n-1)
 - halving until integer reaches 1: myFunc(n/2)
 - processing array/string elements elem by elem: myFunc(arr[i+1]), myFunc(str[i+1])
 - processing SList nodes: myFunc(list.next)
 - Outline: understand func → note return type → RV of orig input → RV of smaller input → connect

Static Books (ELO1)

```

class Book {
    public String title;
    public Library library;
    public static Book last = null;
    public Book(String name) {
        title = name;
        last = this;
    }
    public static String lastBookTitle() {
        return last.title;
    }
    public String getTitle() {
        return title;
    }
}

class Library {
    public Book[] books;
    public int index;
    public static int totalBooks = 0;
    public Library(int size) {
        books = new Book[size];
        index = 0;
    }
    public void addBook(Book book) {
        books[index] = book;
        index++;
        totalBooks++;
        book.library = this;
    }
}
  
```

Annotations:
 - **Static** library, last, totalBooks
 - **Final** lastBookTitle
 - **Instance** ok!
 - **Can't be accessed** (lastBookTitle)
 - **Can be iterated over** (books)
 - **Not allowed, but can't access instance var.** (index)
 - **Instance can access instance** (totalBooks, book.library = this)
 - **bad practice** (totalBooks++)

IntList of (Fa22)

```

public static IntList of(int... x) {
    IntList l = null;
    for (int i = x.length-1; i >= 0; i--) {
        l = new IntList(x[i], l);
    }
    return l;
}
  
```

Annotation: LL → ... BAD; start from end!!

LL → ... BAD; start from end!!
 → ... more efficient
 → ...

Partition (ELO2)

```

public static IntList[] partition(IntList list, int k) {
    IntList[] array = new IntList[k];
    int index = 0;
    IntList l = reverse(list);
    while (l != null) {
        IntList prevAtIndex = array[index];
        IntList next = l.rest;
        array[index] = l;
        array[index].rest = prevAtIndex;
        l = next;
        index = (index+1) % array.length;
    }
    return array;
}
  
```

Diagram: array [] [] []
 → [] [] []

CompareTo/Compare

```

public class Cat implements Comparable<Cat> {
    ...
    @Override
    public int compareTo(Cat o) {
        if (this.getAge() > o.getAge()) {
            return 1;
        } else if (this.getAge() == o.getAge()) {
            return 0;
        }
        return -1;
    }
}

public class CatComparator implements Comparator<Cat> {
    public int compare(Cat c1, Cat c2) {
        return c1.size - c2.size;
    }
}
  
```

OH Queue

```

public class OHQueue implements Iterator<OHRequest> {
    private OHRequest curr;
    private OHRequest request;
    public OHQueue(OHRequest request) {
        curr = request;
    }
    public static boolean isGood(String description) {
        ...
    }
    public boolean hasNext() {
        while (curr != null && !isGood(curr.description)) {
            curr = curr.next;
        }
        return curr != null;
    }
    @Override
    public OHRequest next() {
        if (!hasNext()) {
            throw new NoSuchElementException();
        }
        OHRequest temp = curr;
        curr = curr.next;
        return temp;
    }
}
  
```

Annotations:
 - **OH iterates**
 - **can be iterated over**
 - **can be accessed**
 - **can't be accessed**
 - **bad practice** (curr = request)

IntList replaceValue (Fa22)

```

public static IntList replaceValue(IntList l, int x, int pos) {
    if (l == null) {
        return null;
    }
    if (pos < 0) {
        return null;
    }
    return new IntList(l.first, replaceValue(l.rest, x, pos-1), l.last);
}
  
```

Annotation: replaces if right index

↑ copies as usual

StudentID (Sp23)

```

public class StudentID {
    public String name;
    public int idNumber;
    public Set<String> buildingAccess;
    public static Map<Integer, String> studentDir = new HashMap<>();
    public StudentID(String name, int idNumber) {
        this.name = name;
        this.idNumber = idNumber;
        studentDir.put(idNumber, name);
        buildingAccess = new HashSet<>();
    }
    public static String getStudentName(int idNumber) {
        if (studentDir.containsKey(idNumber)) {
            return studentDir.get(idNumber);
        }
        return "";
    }
}
  
```

OilBomber (Sp23)

```

public void rewritingWorld(int[][] world) {
    Set<Integer> rows = new HashSet<>();
    Set<Integer> cols = new HashSet<>();
    for (int x=0; x < world.length; x++) {
        for (int y=0; y < world[x].length; y++) {
            if (world[x][y] == -1) {
                rows.add(x);
                cols.add(y);
            }
        }
    }
    for (int x=0; x < world.length; x++) {
        for (int y=0; y < world[x].length; y++) {
            if (rows.contains(x) || cols.contains(y)) {
                world[x][y] = 0;
            }
        }
    }
}
  
```

Annotation: trigger

2D Array Exercise (CHKN Sp24)

```

public int[][] reverseInner(int[][] arr) {
    int numRows = arr.length;
    int numCols = arr[0].length;
    int[][] reversedArr = new int[numRows][numCols];
    for (int r=0; r < numRows; r++) {
        for (int c=0; c < numCols; c++) {
            reversedArr[r][c] = arr[r][numCols-1-c];
        }
    }
    return reversedArr;
}
  
```

Annotation: reversed it, goes from end to beg.

Nonagon (Fa22)

```

public int countAdjacentShaded(boolean[][] shaded, int i, int j) {
    int result = 0;
    if (i > 0 && shaded[i-1][j]) result++;
    if (j > 0 && shaded[i][j-1]) result++;
    if (i < shaded.length-1 && shaded[i+1][j]) result++;
    if (j < shaded[0].length-1 && shaded[i][j+1]) result++;
    return result;
}
  
```

Diagram: 3x3 grid with center cell shaded.

TwoWay Street (Sp23)

```

public class TelegraphAve {
    public CrossStreet sentine;
    ...
    public void addPrevStreets() {
        addPrevStreetsHelper(sentine);
    }
    public void addPrevStreetsHelper(CrossStreet curr) {
        if (curr.nextStreet.prevStreet != null) {
            curr.nextStreet.prevStreet = curr;
            addPrevStreetsHelper(curr.nextStreet);
        }
    }
}
  
```

Card Deck (Fa23)

```

private void createDeck() {
    for (int i=0; i < card.suits.length; i++) {
        for (int j=0; j < card.ranks.length; j++) {
            card.add(new Card(i, j));
        }
    }
}
  
```

Annotation: array of suits & ranks

getEveryNth (Sp23)

```

public List<T> getEveryNth(int n) {
    List<T> result = new ArrayList<>();
    for (int i=0; i < this.size(); i=i+n) {
        result.add(array[i]);
    }
    return result;
}

public List<T> getEveryNth(int m) {
    List<T> result = new ArrayList<>();
    Node node = sentinel.next;
    int counter = 0;
    while (node != sentinel) {
        if (counter == 0) {
            result.add(node.value);
            counter = (counter+1) % n;
        }
        node = node.next;
    }
    return result;
}
  
```

Diagram: arraylist [1, 2, 3, 4, 5, 6, 7] → [8, 8, 4]

addPairwise (CHKN Sp24)

```

public static DList addPairwise(DList input) {
    DListNode front = input.sentinel.next;
    DListNode back = input.sentinel.prev;
    DList result = new DList();
    while (front != back && front.prev != back) {
        result.add(front.item + back.item);
        front = front.next;
        back = back.prev;
    }
    if (front == back) {
        result.addLast(front.item);
    }
    return result;
}
  
```

Card Shuffle (Fa23)

```

public void fanShuffle() {
    List<Card> cardCopy = new ArrayList<>(card);
    for (int i=0; i < cardCopy.size(); i++) {
        this.cards.set(i*2, cardCopy.get(i));
        this.cards.set(i*2+1, cardCopy.get(i+1));
    }
}
  
```

Annotation: 2nd hole of cards

I signed an NDA (Fa23)

```

public class NDArray {
    public int value;
    public int dimension;
    public NDArray[] arr;
    public NDArray(int D, int W) {
        dimension = D;
        if (D == 0) {
            return;
        }
        arr = new NDArray[W];
        for (int i=0; i < W; i++) {
            arr[i] = new NDArray(D-1, W);
        }
    }
    public int get(List<Integer> coords) {
        if (coords.size() != dimension) {
            throw new IllegalArgumentException();
        }
        return arr[coords.get(0)].get(coords.subList(1, coords.size()-1));
    }
}
  
```

Annotation: recursively create sub arrays

Dedup (Sp17)

```

public class IntList {
    ...
    public static void removeDuplicates(IntList p) {
        if (p == null) {
            return;
        }
        IntList current = p.rest;
        IntList previous = p;
        while (current != null) {
            if (current.first == previous.first) {
                previous.rest = current.rest;
            } else {
                previous = current;
                current = current.rest;
            }
        }
    }
}
  
```

Annotation: skip over

IntList of (Fa22)

```

public static IntList of(int... x) {
    IntList l = null;
    for (int i = x.length-1; i >= 0; i--) {
        l = new IntList(x[i], l);
    }
    return l;
}
  
```

Annotation: more efficient

ADT

- know WHAT they do, NOT how (interface/abstract)
- List: (ArrayList, LinkedList) ordered, allow duplicates
- Map: (HashMap, TreeMap) key w/ value pair, no key dups
- Set: (HashSet, TreeSet) unordered, no duplicates
- Queue: FIFO
- Stack: LIFO

Disjoint Sets

- public interface DisjointSet {
void connect(x,y) // connect nodes x,y
boolean isConnected(x,y) // true if "connected"
- QuickFind = array of integers (its set)
QuickUnion = stores parent of each node & merges by changing parent
WQU = same as QU, merges smaller into larger → less stringy
WQU w/ Path Comp = sets parent of node to set's root when isConnected() called
- Constructor connect() isConnected()
- | | | | |
|-----------|--------|-------------|-------------|
| QF | $O(N)$ | $O(N)$ | $O(1)$ |
| QU | $O(N)$ | $O(N)$ | $O(N)$ |
| WQU | $O(N)$ | $O(\log N)$ | $O(\log N)$ |
| WQU w/ PC | $O(N)$ | $O(\log N)$ | $O(\log N)$ |
- Path Comp: $isConnected(0,4)$

BST

- quickly access elems in sorted order
 - Node serves as root for smaller tree
 - Node in left subtree < root
 - Node in right subtree > root
- Insert as leaves
Start from root < root → move left
> root → move right
Create new node when we hit null
* traverse: use nodes, left/right pointers
- Hibbard deletion
no children → remove node
1 child → child replaces deleted node (recurse until leaf)
2 child → replace w/ leftmost node on right subtree or rightmost on left

B-Trees / 2-3 trees

- each node up to 2 items, 3 children
 - insert: into existing node
 - < all value → left
 - > all value → right
 - btwn → middle
 - $\Theta(\log N)$ to find node b/c BUSHY
- Complete: $2^N - 1$ elems
 $N = \text{level}$
- Balanced!
- split & push up
- 1 elem = 2 child
2 elem = 3 child

Trees

- N nodes, N-1 edges
- 1 path from root to a node
- fully connected, no cycles
- parent → child
- root = no parent
- leaf = no child

Graphs

- allow cycles
- simple graph = no ll edges, no self edges
- vertices (nodes) & edges
- directed (→), undirected (↔), weights
- degree = # of edges touching it (in/out)
- connected if path btwn every pair of vertices

BFS (level order)

- level based w/ a queue (level by level) 1...2 edge...
 - BFSQ: Add & root to queue while queue not empty: Pop node from front & visit for each IMMEDIATE neighbor: add child to queue if not visited)
- dist to all items on queue = 0 or 1
- Pre: ABCDFEG
Post: FCBEDAG

- GO: A → B → C → F, & return F → C → B
- GO: A → D → E, & return E → D → A
- GO: G & return G

SPT (Shortest Paths)

- BFS = fewest edges, but not shortest
- Dijkstra's: SPT: $O((E+V) \log V)$
- Problem: SPT: shortest path from one to all vertices
- know path = store pointers to prev. nodes
- non-negative edge weights
- explore in increasing distance w/ path length
- PA: add(source, 0)
- for other vertices v, PA.add(v, infinity)
- while PA not empty: p = PQ.removeSmallest()
- relax all edges from p ← update
- Relax edge: p → q w/ weight w: if $\text{distTo}[p] + w < \text{distTo}[q]$:
↳ $\text{distTo}[q] = \text{distTo}[p] + w$
↳ $\text{edgeTo}[q] = p$
↳ PA.changePriority(q, $\text{distTo}[q]$)

Topological Sort

- on DAG (directed acyclic graph) = no cycles
- DFS traversal from every vertex w/ indegree 0 (no parent) & don't clear → reverse DFS-Postorder

Asymptotics

- $O(1) < O(\log N) < O(N) < O(N \log N) < O(N^2) < O(N^3) < O(N^4)$
- Big O: upperbound, grow $\leq f(x)$
- Big Ω : lowerbound, grow $\geq f(x)$
- Big Θ : tightest, upper/lower converge
- Arithmetic sum: ↑ constant factor
↳ $1 + 2 + 3 + \dots + n \in \Theta(n^2)$
↳ $1 + 3 + \dots + \log(n) \in \Theta(\log^2 n)$
- Geometric sum: ↑ multiplicative factor
↳ $1 + 2 + 4 + 8 + \dots + n \in \Theta(n)$
↳ $1 + 3 + 9 + 27 + \dots + n \log(n) \in \Theta(n \log n)$
- * $2 + 2^2 + 2^3 + \dots + N^2 \in \Theta(N^3)$
- constant sum:
↳ $1 + 1 + \dots + 1 \in \Theta(n)$ → $\Theta(\text{num elem})$ $N + N + N + \dots + N = N^2$

DFS

- visit each subtree recursively w/ a stack
- (no in-order for graphs)
- Tree Traversal:
 - Pre-order: cross LEFT
Print(x.key)
preorder(left)
preorder(right)
 - mark parent then child
 - visit, go left, go right
 - DBCFEG

General Graph Traversals

- BFS = in order of distance
 - Pre-order = visit, go to children
↳ "visit node as soon as it enters the stack" myself → my children
 - post-order = go to children, visit
↳ "visit node once it leaves the stack": my children → myself
- DFS (start):
preorder.add(start)
visited.add(start)
for each neighbor of start:
if not visited: DFS(neighbor)
postorder.add(start)

MST (Minimum Spanning Tree)

- MST = lowest edge weight sum - acyclic, connected, all vertices of G
 - Prim's: start w/ (any) node
add shortest edge that connects one node in MST to a node outside MST
repeat until V-1 edges
 - Cut property: given any cut, any min. weight crossing edge is in MST
 - Kruskal's:
 - maintain edges set for MST
 - consider ea. edge in increasing order
a) add to MST that doesn't create cycle
 - repeat until V-1 edges
- Unique edge weights = 1 MST
- * $E \log V$
- * sort/iterate or put in PQ
- * WQU for cycles

CT (Method Signature & Lock in ST)

- RT: can be more specific method (or) check parameter type
- Reminders:
 - int[] zero = new int[3]
 - List<String> list = new ArrayList<>()
 - Set<Integer> set = new HashSet<>()
 - for (int i = 0; i < s.length(); i++)
 - char letter = s.charAt(i)
 - char c = 'a'
 - String s = "hello"

Asymptotics

1) $f(n) \in O(g(n))$ & $g(n) \in \Omega(f(n))$ in creating

$f(n) \in O(g(n))$
 $f(n) \in \Omega(g(n))$
 $f(n) \in \Omega(\log(g(n)))$ } $\log(g(n)) < g(n) < g(n) \cdot n$
 $f(n) \in O(n \cdot g(n))$

Percolation (F23 mt2)

public boolean topBottomConnected(Hole[] holes) {
 WeightedQuickUnion q = new WeightedQuickUnion(holes.length+2);
 int bottom = holes.length-1; // virt. node
 int top = holes.length+1; // virt. node
 // are they connected?
 for (int i=0; i<holes.length; i++) {
 if (hole.get(i).intersectsBottom())
 q.union(i, bottom);
 if (hole.get(i).intersectsTop())
 q.union(i, top);
 }
 for (int i=0; i<holes.length; i++)
 for (int j=0; j<holes[i].size(); j++)
 if (holes[i].get(j).intersectsWithHole(holes[i+1].get(j)))
 q.union(i, i+1);
 return q.isConnected(top, bottom);
}

Finish Runtimes

$\Theta(N^2)$

for (int i=1; i<N; i++)
 for (int j=1; j<N; j++)
 System.out.println("hi")

1) $g(N) = N$
 2) $i=i+1$
 3) $f(i)=i$
 4) $f(i)+f(i-1)+...+f(1)$
 $1+2+3+...+N = \frac{N(N+1)}{2} \in \Theta(N^2)$

$\Theta(N \log N)$

for (int i=1; i<N; i++)
 for (int j=1; j<N; j++)
 System.out.println(i)

1) $g(N) = N$
 2) $i=i+1$
 3) $f(i)=i$
 4) $f(i)+f(i-1)+...+f(1)$
 $1+2+3+...+N = \frac{N(N+1)}{2} \in \Theta(N^2)$

$\Theta(N^2)$

for (int i=1; i<N; i++)
 for (int j=1; j<N; j++)
 System.out.println(i)

1) $g(N) = N$
 2) $i=i+1$
 3) $f(i)=i$
 4) $f(i)+f(i-1)+...+f(1)$
 $1+2+3+...+N = \frac{N(N+1)}{2} \in \Theta(N^2)$

$\Theta(N^2)$

for (int i=1; i<N; i++)
 for (int j=1; j<N; j++)
 System.out.println(i)

1) $g(N) = N$
 2) $i=i+1$
 3) $f(i)=i$
 4) $f(i)+f(i-1)+...+f(1)$
 $1+2+3+...+N = \frac{N(N+1)}{2} \in \Theta(N^2)$

$\Theta(N^2)$

for (int i=1; i<N; i++)
 for (int j=1; j<N; j++)
 System.out.println(i)

1) $g(N) = N$
 2) $i=i+1$
 3) $f(i)=i$
 4) $f(i)+f(i-1)+...+f(1)$
 $1+2+3+...+N = \frac{N(N+1)}{2} \in \Theta(N^2)$

Taiko

• Queue < Equipment
 private Map<String, Chu> chus = new TreeMap<>();
 Set<Shime> shimes = new HashSet<>();
 public void organizeQueue(Taiko equipment)
 while (equipment.size() != 0)
 Taiko = equipment.remove();
 if (it instanceof Chu chu)
 chus.put(chu.getName(), chu);
 else
 shimes.add((Shime) t);

• DMS: chu h = new Chu("h", 7);
 static dynamic
 • CT: allows (chu) s & (chu) m
 bic s & m = static type Taiko
 • Taiko is a chu, do I allow
 • RTE: s & m's DT = Taiko & shime,
 which isn't chu
 • CT: (shime) h, but "h" w/ DT Chu = NO!

Iterator = does the iterating

public interface Iterator<E>
 boolean hasNext(); // another elem to iterate over?
 E next(); // give next elem

Iterable = gets iterated over
 public interface Iterable<E> // public iterator <Dog> iterator()
 Iterator<E> iterator(); // return iterator over iterable's elem

Comparable = to compare types
 Comparators = compare 2 obj.
 @ obj less, @ obj =, @ obj greater

CompareTo/compare

public class CatComparator implements Comparator<Cat>
 public int compare(Cat a, Cat b)
 return a.size() - b.size;

public class Cat implements Comparable
 @Override
 public int compareTo(Cat a)
 if (this.getAge() > a.getAge()) return 1
 else if (this.getAge() == a.getAge()) return 0
 return -1

4 MT2 J2F

• Public class PrimeFactorIterator implements
 Iterator<Integer>, Iterable<Integer>
 private int x;
 public PrimeFactorIterator(int givenX)
 this.x = givenX; // check for req. on next & ret
 public boolean hasNext()
 return x >= 2

q3(w) $\log N$
q3(N) $\log N^2$
q3(N^2) $\log N^4$
q3(N^4) $\log N^8$
q3(N^8) $\log N^{16}$
q3(N^{16}) $\log N^{32}$
q3(N^{32}) $\log N^{64}$
q3(N^{64}) $\log N^{128}$
q3(N^{128}) $\log N^{256}$
q3(N^{256}) $\log N^{512}$
q3(N^{512}) $\log N^{1024}$
q3(N^{1024}) $\log N^{2048}$
q3(N^{2048}) $\log N^{4096}$
q3(N^{4096}) $\log N^{8192}$
q3(N^{8192}) $\log N^{16384}$
q3(N^{16384}) $\log N^{32768}$
q3(N^{32768}) $\log N^{65536}$
q3(N^{65536}) $\log N^{131072}$
q3(N^{131072}) $\log N^{262144}$
q3(N^{262144}) $\log N^{524288}$
q3(N^{524288}) $\log N^{1048576}$
q3(N^{1048576}) $\log N^{2097152}$
q3(N^{2097152}) $\log N^{4194304}$
q3(N^{4194304}) $\log N^{8388608}$
q3(N^{8388608}) $\log N^{16777216}$
q3(N^{16777216}) $\log N^{33554432}$
q3(N^{33554432}) $\log N^{67108864}$
q3(N^{67108864}) $\log N^{134217728}$
q3(N^{134217728}) $\log N^{268435456}$
q3(N^{268435456}) $\log N^{536870912}$
q3(N^{536870912}) $\log N^{1073741824}$
q3(N^{1073741824}) $\log N^{2147483648}$
q3(N^{2147483648}) $\log N^{4294967296}$
q3(N^{4294967296}) $\log N^{8589934592}$
q3(N^{8589934592}) $\log N^{17179869184}$
q3(N^{17179869184}) $\log N^{34359738368}$
q3(N^{34359738368}) $\log N^{68719476736}$
q3(N^{68719476736}) $\log N^{137438953472}$
q3(N^{137438953472}) $\log N^{274877906944}$
q3(N^{274877906944}) $\log N^{549755813888}$
q3(N^{549755813888}) $\log N^{1099511627776}$
q3(N^{1099511627776}) $\log N^{2199023255552}$
q3(N^{2199023255552}) $\log N^{4398046511104}$
q3(N^{4398046511104}) $\log N^{8796093022208}$
q3(N^{8796093022208}) $\log N^{17592186044416}$
q3(N^{17592186044416}) $\log N^{35184372088832}$
q3(N^{35184372088832}) $\log N^{70368744177664}$
q3(N^{70368744177664}) $\log N^{140737488355328}$
q3(N^{140737488355328}) $\log N^{281474976710656}$
q3(N^{281474976710656}) $\log N^{562949953421312}$
q3(N^{562949953421312}) $\log N^{1125899906842624}$
q3(N^{1125899906842624}) $\log N^{2251799813685248}$
q3(N^{2251799813685248}) $\log N^{4503599627370496}$
q3(N^{4503599627370496}) $\log N^{9007199254740992}$
q3(N^{9007199254740992}) $\log N^{18014398509481984}$
q3(N^{18014398509481984}) $\log N^{36028797018963968}$
q3(N^{36028797018963968}) $\log N^{72057594037927936}$
q3(N^{72057594037927936}) $\log N^{144115188075855872}$
q3(N^{144115188075855872}) $\log N^{288230376151711744}$
q3(N^{288230376151711744}) $\log N^{576460752303423488}$
q3(N^{576460752303423488}) $\log N^{1152921504606846976}$
q3(N^{1152921504606846976}) $\log N^{2305843009213693952}$
q3(N^{2305843009213693952}) $\log N^{4611686018427387904}$
q3(N^{4611686018427387904}) $\log N^{9223372036854775808}$
q3(N^{9223372036854775808}) $\log N^{18446744073709551616}$
q3(N^{18446744073709551616}) $\log N^{36893488147419103232}$
q3(N^{36893488147419103232}) $\log N^{73786976294838206464}$
q3(N^{73786976294838206464}) $\log N^{147573952589676412928}$
q3(N^{147573952589676412928}) $\log N^{295147905179352825856}$
q3(N^{295147905179352825856}) $\log N^{590295810358705651712}$
q3(N^{590295810358705651712}) $\log N^{1180591620717411303424}$
q3(N^{1180591620717411303424}) $\log N^{2361183241434822606848}$
q3(N^{2361183241434822606848}) $\log N^{4722366482869645213696}$
q3(N^{4722366482869645213696}) $\log N^{9444732965739290427392}$
q3(N^{9444732965739290427392}) $\log N^{18889465931478580854784}$
q3(N^{18889465931478580854784}) $\log N^{37778931862957161709568}$
q3(N^{37778931862957161709568}) $\log N^{75557863725914323419136}$
q3(N^{75557863725914323419136}) $\log N^{151115727451828646838272}$
q3(N^{151115727451828646838272}) $\log N^{302231454903657293676544}$
q3(N^{302231454903657293676544}) $\log N^{604462909807314587353088}$
q3(N^{604462909807314587353088}) $\log N^{1208925819614629174706176}$
q3(N^{1208925819614629174706176}) $\log N^{2417851639229258349412352}$
q3(N^{2417851639229258349412352}) $\log N^{4835703278458516698824704}$
q3(N^{4835703278458516698824704}) $\log N^{9671406556917033397649408}$
q3(N^{9671406556917033397649408}) $\log N^{19342813113834066795298816}$
q3(N^{19342813113834066795298816}) $\log N^{38685626227668133590597632}$
q3(N^{38685626227668133590597632}) $\log N^{77371252455336267181195264}$
q3(N^{77371252455336267181195264}) $\log N^{154742504910672534362390528}$
q3(N^{154742504910672534362390528}) $\log N^{309485009821345068724781056}$
q3(N^{309485009821345068724781056}) $\log N^{618970019642690137449562112}$
q3(N^{618970019642690137449562112}) $\log N^{1237940039285380274899124224}$
q3(N^{1237940039285380274899124224}) $\log N^{2475880078570760549798248448}$
q3(N^{2475880078570760549798248448}) $\log N^{4951760157141521099596496896}$
q3(N^{4951760157141521099596496896}) $\log N^{9903520314283042199192993792}$
q3(N^{9903520314283042199192993792}) $\log N^{19807040628566084398385987584}$
q3(N^{19807040628566084398385987584}) $\log N^{39614081257132168796771975168}$
q3(N^{39614081257132168796771975168}) $\log N^{79228162514264337593543950336}$
q3(N^{79228162514264337593543950336}) $\log N^{158456325028528675187087900672}$
q3(N^{158456325028528675187087900672}) $\log N^{316912650057057350374175801344}$
q3(N^{316912650057057350374175801344}) $\log N^{633825300114114700748351602688}$
q3(N^{633825300114114700748351602688}) $\log N^{1267650600228229401496703205376}$
q3(N^{1267650600228229401496703205376}) $\log N^{2535301200456458802993406410752}$
q3(N^{2535301200456458802993406410752}) $\log N^{5070602400912917605986812821504}$
q3(N^{5070602400912917605986812821504}) $\log N^{10141204801825835211973625643008}$
q3(N^{10141204801825835211973625643008}) $\log N^{20282409603651670423947251286016}$
q3(N^{20282409603651670423947251286016}) $\log N^{40564819207303340847894502572032}$
q3(N^{40564819207303340847894502572032}) $\log N^{81129638414606681695789005144064}$
q3(N^{81129638414606681695789005144064}) $\log N^{162259276829213363391578010288128}$
q3(N^{162259276829213363391578010288128}) $\log N^{324518553658426726783156020576256}$
q3(N^{324518553658426726783156020576256}) $\log N^{649037107316853453566312041152512}$
q3(N^{649037107316853453566312041152512}) $\log N^{1298074214633706907132624082305024}$
q3(N^{1298074214633706907132624082305024}) $\log N^{2596148429267413814265248164610048}$
q3(N^{2596148429267413814265248164610048}) $\log N^{5192296858534827628530496329220096}$
q3(N^{5192296858534827628530496329220096}) $\log N^{10384593717069655257060992658440192}$
q3(N^{10384593717069655257060992658440192}) $\log N^{20769187434139310514121985316880384}$
q3(N^{20769187434139310514121985316880384}) $\log N^{41538374868278621028243970633760768}$
q3(N^{41538374868278621028243970633760768}) $\log N^{83076749736557242056487941267521536}$
q3(N^{83076749736557242056487941267521536}) $\log N^{166153499473114484112975882535043072}$
q3(N^{166153499473114484112975882535043072}) $\log N^{332306998946228968225951765070086144}$
q3(N^{332306998946228968225951765070086144}) $\log N^{664613997892457936451903530140172288}$
q3(N^{664613997892457936451903530140172288}) $\log N^{1329227995784915872903807060280344576}$
q3(N^{1329227995784915872903807060280344576}) $\log N^{2658455991569831745807614120560689152}$
q3(N^{2658455991569831745807614120560689152}) $\log N^{5316911983139663491615228241121378304}$
q3(N^{5316911983139663491615228241121378304}) $\log N^{10633823966279326983230456482242756608}$
q3(N^{10633823966279326983230456482242756608}) $\log N^{21267647932558653966460912964485513216}$
q3(N^{21267647932558653966460912964485513216}) $\log N^{42535295865117307932921825928971026432}$
q3(N^{42535295865117307932921825928971026432}) $\log N^{85070591730234615865843651857942052864}$
q3(N^{85070591730234615865843651857942052864}) $\log N^{170141183460469231731687303715884105728}$
q3(N^{170141183460469231731687303715884105728}) $\log N^{340282366920938463463374607431768211456}$
q3(N^{340282366920938463463374607431768211456}) $\log N^{680564733841876926926749214863536422912}$
q3(N^{680564733841876926926749214863536422912}) $\log N^{1361129467683753853853498429727072845824}$
q3(N^{1361129467683753853853498429727072845824}) $\log N^{2722258935367507707706996859454145691648}$
q3(N^{2722258935367507707706996859454145691648}) $\log N^{5444517870735015415413993718908291383296}$
q3(N^{5444517870735015415413993718908291383296}) $\log N^{10889035741470030830827987437816582766592}$
q3(N^{10889035741470030830827987437816582766592}) $\log N^{21778071482940061661655974875633165533184}$
q3(N^{21778071482940061661655974875633165533184}) $\log N^{43556142965880123323311949751266331066368}$
q3(N^{43556142965880123323311949751266331066368}) $\log N^{87112285931760246646623899502532662132736}$
q3(N^{87112285931760246646623899502532662132736}) $\log N^{174224571863520493293247799005065324265472}$
q3(N^{174224571863520493293247799005065324265472}) $\log N^{348449143727040986586495598010130648530944}$
q3(N^{348449143727040986586495598010130648530944}) $\log N^{696898287454081973172991196020261297061888}$
q3(N^{696898287454081973172991196020261297061888}) $\log N^{1393796574908163946345982392040522594123776}$
q3(N^{1393796574908163946345982392040522594123776}) $\log N^{2787593149816327892691964784081045188247552}$
q3(N^{2787593149816327892691964784081045188247552}) $\log N^{5575186299632655785383929568162090376495104}$
q3(N^{5575186299632655785383929568162090376495104}) $\log N^{11150372599265311570767859136324180752990208}$
q3(N^{11150372599265311570767859136324180752990208}) $\log N^{22300745198530623141535718272648361505980416}$
q3(N^{22300745198530623141535718272648361505980416}) $\log N^{44601490397061246283071436545296723011960832}$
q3(N^{44601490397061246283071436545296723011960832}) $\log N^{89202980794122492566142873090593446023921664}$
q3(N^{89202980794122492566142873090593446023921664}) $\log N^{178405961588244985132285746181186892047843328}$
q3(N^{178405961588244985132285746181186892047843328}) $\log N{356811923176489970264571492362373784095686656}$
q3(N^{356811923176489970264571492362373784095686656}) $\log N{7136238463529799405291429847247475681913$

① Sorting

	memory	RT (best)	RT (Worst)	Stable	Notes
Selection	$\Theta(1)$	$\Theta(N^2)$	$\Theta(N^2)$	N	
Insertion	$\Theta(1)$	$\Theta(N)$	$\Theta(N^2)$	Y	fastest for small N & almost sorted
Mergesort	$\Theta(N)$	$\Theta(N \log N)$	$\Theta(N \log N)$	Y	fastest stable sort
Heapsort	$\Theta(1)$	$\Theta(N \log N)$	$\Theta(N \log N)$	N	Bad cache
Quicksort	$\Theta(1)$ (Hoare) $\Theta(N \log N)$ (3-way)	$\Theta(N \log N)$	$\Theta(N^2)$	$\Theta(N)$ (Hoare) Y (3-way)	Fastest compare sort
Counting	$\Theta(N+R)$	$\Theta(N+R)$		Y	alphabet keys only
LSD	$\Theta(N+R)$	$\Theta(WN+WR)$		Y	strings of alphabetical keys only
MSD	$\Theta(N+WR)$	$\Theta(N+R)$	$\Theta(WN+WR)$	Y	Bad cache

N: # of keys, input size
R: size of alphabet
W: width of longest key
X: assumes constant CompareTo time

• stability: order of equivalent items are preserved
'don't cross over when stably sorted'

• inversion: pair of unordered elems

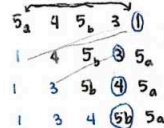
Ex: [8, 5, 7, 2] → 5 inversions

Stable: 2, 3, 5, 6, 7
Unstable: 2, 3, 5, 6, 7

* Comparison sort: at least $N \log N$ comparisons

② Sorts

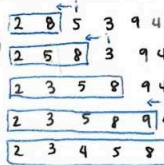
• Selection



• find smallest item in unsorted portion of array
• move it to end of sorted portion of array

* Front items sorted first
* smallest elem chosen & being swapped

• Insertion (in-place)

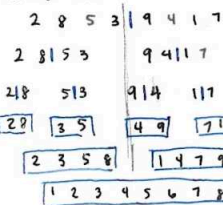


* unsorted elem just shifts
* item i as traveling item

• swap item backwards until i is in right place among prev examined

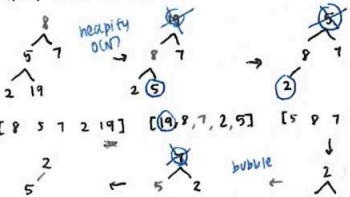
• repeat until i is last item

• Merge Sort



• split in half
• merge 2 sorted half
• recursively sort halves
* elem in left 'tree' doesn't interact w/ right 'tree' until the end

• Heap Sort (in-place)

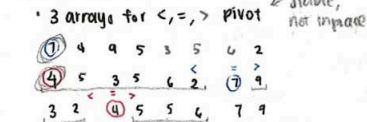


• heapify array ($\Theta(N)$)
• swap top of heap w/ last elem in unsorted portion
• bubble-down new top
* max sorted first
* work backwards

• Quick Sort

* look for pivot!
• select a pivot
• everything less, goes left
• everything greater, goes right

• 3-Way Partition



• Hoare Partition

• 2 pointers on op ends
• Move towards each other
• swap if inversion detected
• repeat until pointers cross
* random pivot, shuffle, median

• First Item = pivot

• 2 pointers

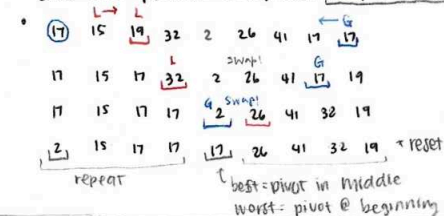
• L likes small, starts at left
• G likes big, starts at right

• move pointers towards ea other & stop @ dublike

• L dislikes $\geq$ items
• G dislikes $\leq$ items

• both stop → swap & move pointers

• done when pointers cross, then swap G & pivot



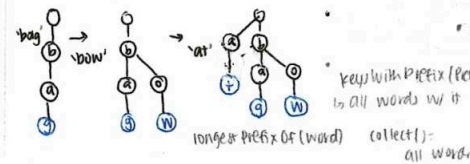
③ Tries

• specialized trees for specific string

• nodes store one letter

• nodes are shared amongst keys (words)

• mark nodes = blue to represent word-ends



• RT: independent of # of keys in trie

• L: length of word operated on

• add (word)

• $\Theta(L)$ w/ respect to # of keys in trie
• $\Theta(L)$ w/ respect to length of word

• contains (word)

• $\Theta(L)$ w/ respect to # of keys in trie
• $\Theta(L)$ w/ respect to length of word

• fails: final word ≠ word-end, search falls off tree

④ Radix Sort

• radix: base of a numbering system, alphabet to sort into.

• Counting sort: subroutine, digit-by-digit

• doesn't use comparisons

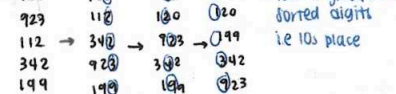
• counts occurrences of each item in count array

• calculate starting pos. of each elem using count array

* directly places in elem

• LSD Radix

• least sig. digit = rightmost start

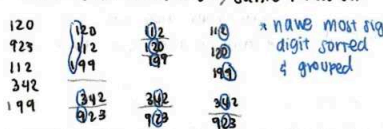


• MSD Radix

• most sig. digit = leftmost start

• best case: one scan, MSDs all unique

Worst: scan every digit, same RT as LSD



⑤ SPT & MST

Problem	Alg.	RT (if $\epsilon > 0$)	Notes
Shortest Paths	Dijkstra	$\Theta(E \log V)$	fails for Θ edge weights
MST	Prim's	$\Theta(E \log V)$	analogous to Dijkstra
MST	Kruskal	$\Theta(E \log E)$	use woupc
MST	Kruskal w/ pre-sorted edges	$\Theta(E \log V)$	use woupc

LSD Sort (list):

not in-place
find max # of digits of a # in list
for ea. digit, place starting from 0 to max # of digits (i.e. right to left):
counting sort (list, current digit place)

⑥ Algorithm Design

• steps to solve problem efficiently (ie graph/sorting)
• if problem similar to problem solved by algorithm we know
↳ similar output, runtime... → modify to fit specs
• manipulate input to make solving easier
↳ remove certain edges, make graph directed
• break problem into steps
↳ sort only part of list

⑦ Pseudocode

insertionSort(list) ← in-place
for ea. elem in list:
while curr elem is < prev elem:
swap curr & prev elem
selectionSort(list) ← in-place
for ea. elem in list:
find min elem in rest of list
swap curr elem & min elem

mergeSort(list):

← not in-place
if list.length == 0 || list.length == 1:
return list
left = left half of list
right = right half of list
left = mergeSort(left)
right = mergeSort(right)
return merge(left, right)

heapSort(list):

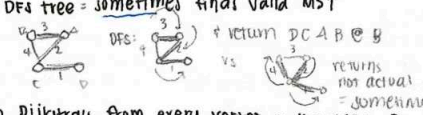
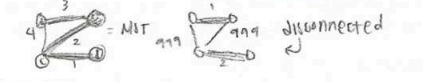
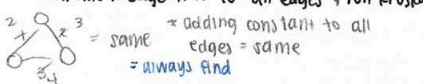
← in-place
heapify list into max-heap
while max-heap is not empty:
swap root w/ last unsorted elem
bubbleDown (new root)

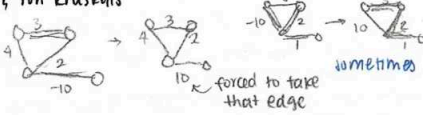
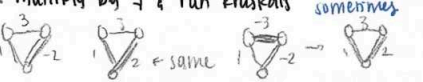
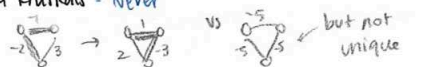
QuickSort(list):

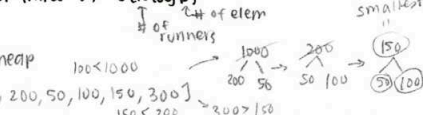
Choose pivot
partition elem abt. pivot (<, =, >)
return quickSort(<) + pivot(=) + quickSort(>)

* LSD: # of passes = same for # of digits
* grouping by LSD or MSD bins

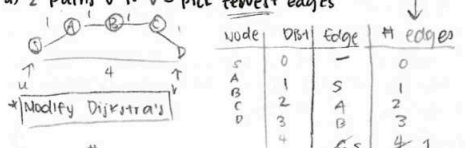
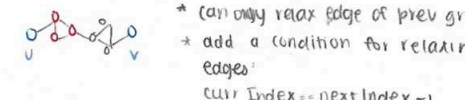
TR55

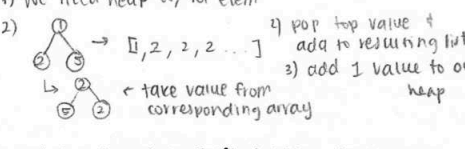
- Q6 MST Adaptations
- graph = simple, connected, undirected, ? cycle, weights of all edges distinct
 - at least 1 θ edge $\rightarrow$ Kruskal's: always finds a valid MST
 $\rightarrow$ Dijkstra's = SPT would fail
 - graph has 1 cycle $\rightarrow$ DFS & break ties by edge weight
 $\hookrightarrow$ DFS tree = sometimes finds valid MST
- 
- run Dijkstra's from every vertex in the graph for 1 step so that from each vertex = found one edge
 $\hookrightarrow$ add edges to set & output sometimes
- 
- Will modify original graph $\rightarrow$ will it find 0 & MST?
 - add minimum edge (M1) to all edges & run Kruskal's
- 

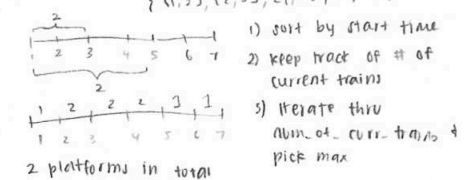
- 2) ≥ 1 negative edge & multiply each θ by -1 & run Kruskal's
- 
- fully connected, ev. vertex is incident to $> \theta$ edge & multiply by -1 & run Kruskal's sometimes
- 
- incident w/ ≥ 1 edge & multiply edge by -1 & Kruskal's = NEVER
- 

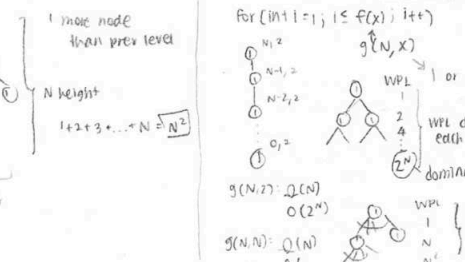
- Q2: Mechanical Sorting
- Call selection sort first w/c it's unstable
- sorting race times for NYC marathon to get 100 fastest times w/ $O(N \log N)$
- 
- max-heap
- 

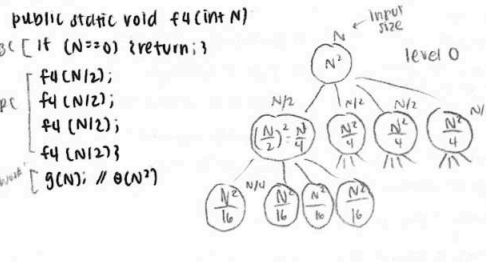
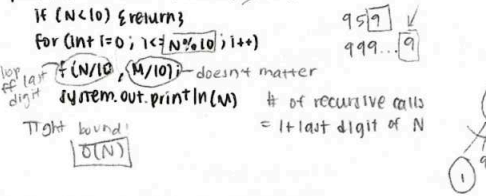
- Q23
- Q1(int N)
- ```
for (int i=1; i<N; i++)
for (int j=i; j<N; j++)
 print(i)
```
- for (j=1; j<N; j++)  $\leftrightarrow$  for (c=N; c>1; c/=2)

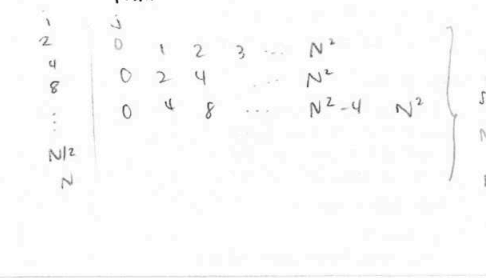
- Q4 Graph Answer
- undirected weight graph,  $\theta$  weights
  - find u to v w/ lowest cost,  $O(E \log V)$
- a) 2 paths u to v - pick fewer edges
- 
- Modify Dijkstra's
- add edges col
  - pick one w/ min # edges
- b) visit s & k along path
- Dijkstra's from u
  - Dijkstra's from s, k
- u  $\rightarrow$  k  $\rightarrow$  s  $\rightarrow$  v - dist 1  
or u  $\rightarrow$  s  $\rightarrow$  k  $\rightarrow$  v - dist 2
- c) split vertices into 5 sep. grps (w/o start & end)  
visit vertices in order of subset
- 

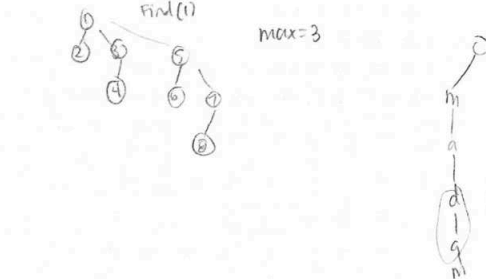
- Q5 Sorting Shenanigans
- M sorted lists of variable lengths into one list.
- $O(N \log M)$
- 1) Algo Design Merge Sort  $N \log N > N \log M \times$   
OS  $N \log N > N \log M \times$  if  $m < N$
- Heapsort  $(N \log N)$
- Heapify to remove largest Min-heap
- If we have  
N elem  $\rightarrow \log N$   
M elem  $\rightarrow \log M$
- We need heap w/ M elem
- 2) 

- Q arrival & departure times of trains, find min # of platforms to avoid delay:  $O(N \log N)$   $N$  # trains
- {[5,7], [1,5], [2,3], [4,3]}
- Start dep. sort by start time: {1,5}, [2,3], [4,3], [5,7]}
- 

- WPL
- 

- 2) Slightly Harder
- public static void f4(int N)
- ```
if (N==0) return;  
f4(N/2);  
f4(N/2);  
f4(N/2);  
f4(N/2);  
g(N); // O(N^2)
```
- 
- work per level
- level 0: N^2
- level 1: $4(\frac{N^2}{4}) = N^2$
- level 2: $16(\frac{N^2}{16}) = N^2$
- $\log_2 N$ height $\rightarrow \boxed{\Theta(N^2 \log N)}$
- public static void f5(int N, int M)
- ```
if (N<10) return;
for (int i=0; i<N%10; i++)
 f(N/10, M/10);
system.out.println(N);
Tight bound: $\boxed{O(N)}$
```
- 

- Q6 Sigma 3 (int N)
- ```
for (int i=2; i<N; i++)  
    for (int j=0; j<N-i; j++)  
        print(i+j)
```
- depend
- 

- VBUPC
- 
- Method: `next = node.children.getOrDefault(substr.charAt(index), null)`
- int result = numEnd of words below
- helper (root, substr, start, length/2)
- if node == null return 0
- if index == substr.length() return node.numEnd of
- TreeNode node.next = node.children.getOrDefault(H4, null)

- level of bot.
- $1+10^1+10^2+\dots+10^{\text{height}}$
- $\text{height} = \log_{10} N$
- $\text{WPL} = 10^{\text{level}} \Rightarrow 10^{\log_{10} N} = N$
- Q7
- for (int i=0; i<N; i++)
 for (int j=i; j<N; j++)
 if (j%2 == 0)
- 